STUDENT FIELD GUIDE

BATTLE BOTS

Program a robot in assembly language.
Then watch it fight.



⚙️ Instructions | 🎯 Missions | 🔍 Debugging | 🎨 Customize | 🔑 Answer key

NAME _____ CLASS _____

01 Welcome, robot programmer!

You write the code. Your robot does the fighting.

In Battle Bots, you write a program that controls a robot tank. Then you press Start and watch it battle other robots. You can't steer with a keyboard or a controller. Once the match starts, **your code is in charge**.

You'll write your program in **assembly language**. It's one of the simplest computer languages there is. Each line is one tiny instruction, like "add 5" or "turn left" or "jump to another line." The chips inside phones, game consoles and cars run on instructions just like these.

🤖 **Fun fact**

Real assembly language is how programmers talk almost directly to a computer chip. You're learning the same ideas, just with a robot!

What's inside

01 **Welcome** — How a match works

02 **Meet the app** — Your screen and tools

03 **Big ideas** — Registers, labels, angles

04 **Every instruction** — Commands, memory and the stack

05 **Sensors** — How your robot "sees"

06 **Missions** — 15 guided missions

07 **Read a real robot** — Wanderer, line by line

08 **Debug like a pro** — Breakpoints and mistakes

09 **Make it yours** — Looks and the final battle

10 **Challenges** — Level up your robot

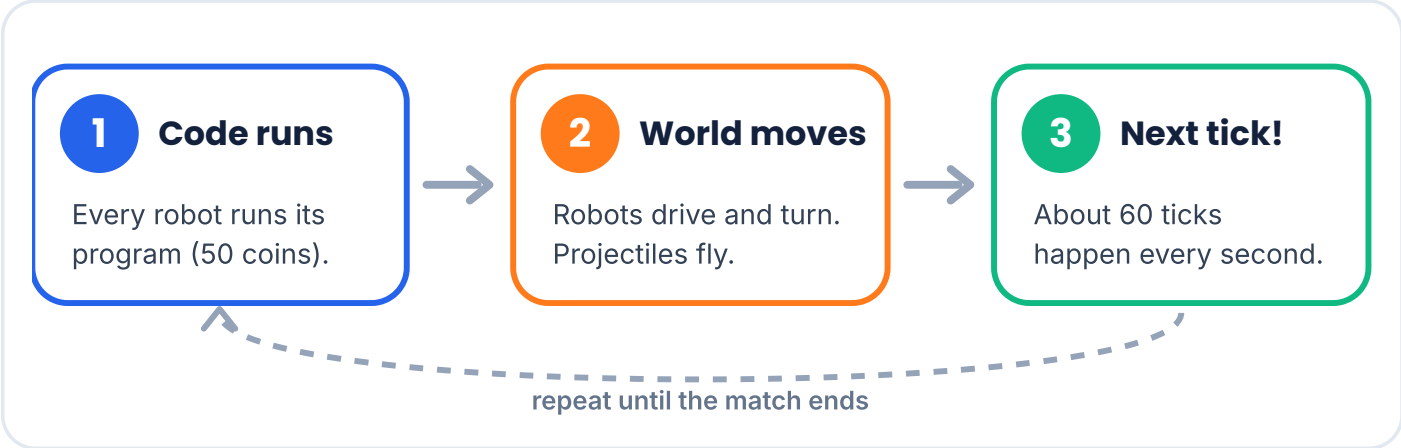
11 **Answer key** — Every challenge, solved

🎮 **You're the coach, not the player**

Once you press Start, you can't help your robot. Everything it does comes from the code you wrote. Plan ahead!

How a match works

The game runs in tiny steps called **ticks**. Every tick, three things happen:



Each robot gets **50 coins** (called cycles) to spend every tick. Most instructions cost 1 coin. `SCAN` and `RADAR` cost 3. Your program reads from top to bottom, one line at a time, unless a **jump** sends it somewhere else. When it reaches the bottom, it starts over at the top.

Winning a match

Arena rules

- Rows of **trees** are obstacles. They stop robots, projectiles and scans.
- Brown patches are **mud**. In mud, robots can only go speed 2.
- Bumping a wall at full speed costs 2 health.

How to win

- Every robot starts with **100 health**.
- Each projectile hit takes away **10**.
- Last robot standing wins! If time runs out (6000 ticks), the most health wins.

Words to know

Instruction
One command, like `ADD` or `FIRE`.

Program
A list of instructions for your robot.

Tick
One tiny step of the game.

Register
A box that holds one number.

Label
A bookmark in your code.

Sensor
How your robot learns about the world.

Function
A chunk of code you can `CALL` from anywhere.

Breakpoint
A red dot that pauses your robot on a line.

Mission
A short challenge inside the app.

The arena



A map of the arena. Notice that (0, 0) is the top-left corner!

Your first robot in 1 minute ⏱️

1 **Click "+ New robot"**
A starter robot opens in the editor.

2 **Replace the code**
Type Spinner exactly.

3 **Press ▶ Start**
It uses your new code for you.

4 **Watch!**
Spinner drives in circles.

spinner.asm

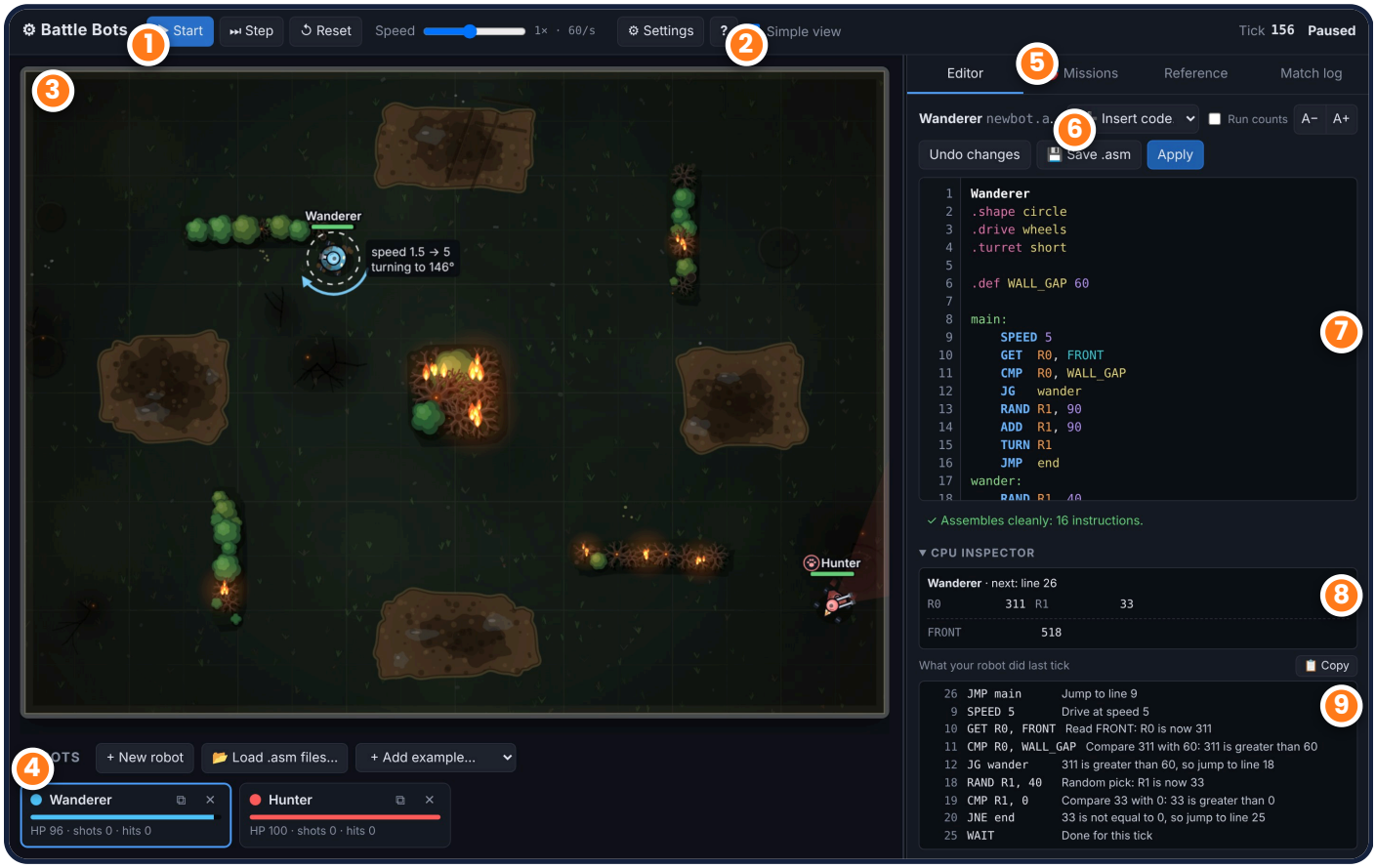
```
Spinner
main:
    SPEED 3      ; medium speed
    TURN 10      ; keep turning right
    WAIT
    JMP main
```

🎯 **Brand new? Start with the missions!**
The first time you open Battle Bots, a welcome box pops up. Click **Start the first mission** to learn one idea at a time, or **Look around first** to explore. Section 06 tells you all about missions.

02 Meet the app

A quick tour of your Battle Bots screen.

Here's what you'll see while a match is paused. Find each number, then read what it does below.



1 Controls
▶ Start, ⏭ Step one tick, ↺ Reset.

2 Simple view
Plain-English help. Keep it on!

3 The arena
Where the robots battle.

4 Robot cards
Each robot's health. Click one to edit it.

5 Tabs
Editor, 🎯 Missions, Reference, Match log.

6 Code helpers
➕ Insert code, Run counts, text size (A− A+).

7 Code editor
Write your program here.

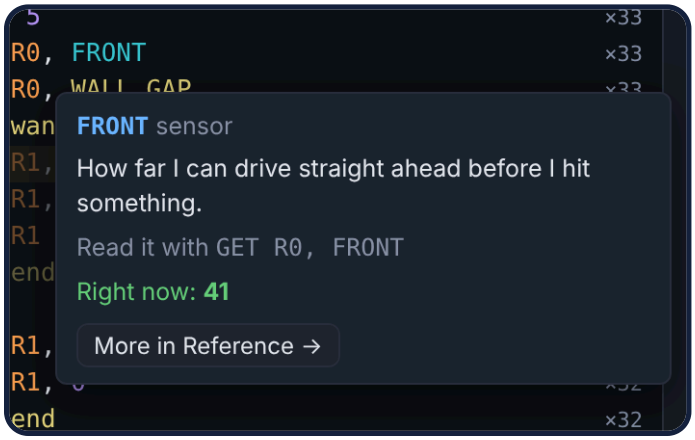
8 CPU inspector
What's in your robot's boxes right now.

9 Last tick
Every line your robot ran, explained.

💡 **▶ Start always runs your newest code**

You don't need to press Apply first. **Start** and **Step** apply your edits for you. If your code has a mistake, nothing runs, and the editor shows you where the mistake is.

Your coding toolbox 🧰



Point at a word for help

Hold your mouse over any instruction, sensor or register in your code. A box pops up that explains it in plain English. For sensors and registers, it even shows the value **right now**!

Want the whole story? Click **More in Reference →**. The **Reference** tab lists every instruction and sensor, with a search box.

✅ Simple view

It's on by default. You get plain-English help, and the CPU inspector shows only the boxes your program uses. Untick it later when you want every detail.

⌨️ Autocomplete

Start typing and suggestions pop up: instructions, sensors and your own labels. Press ↑ ↓ to choose, Enter or Tab to use one, Esc to close.

➕ Insert code: ready-made patterns

Pick a pattern from the ➕ **Insert code...** menu. It drops into your program where your cursor is, with comments that explain it. Then the cursor waits right where **your** code goes.

- **Main loop** — The loop every robot needs.
- **IF (one choice)** — Do something only sometimes.
- **IF / ELSE** — Pick one of two choices.
- **Repeat 5 times** — Count with a register.
- **Function** — Code you CALL, ending in RET.
- **Scan and fire** — Find an enemy and aim at it.
- **Wall check** — Turn before you crash.
- **Dodge projectiles** — Use RADAR to escape.

Keyboard shortcuts

Keys	What they do
Space	Start or pause the match
.	Step: run one tick
R	Reset the arena
Tab / Shift + Tab	Indent or un-indent lines
Ctrl + /	Turn lines into comments, or back
Ctrl + Z	Undo. (The **Undo changes** button goes back to the code you last ran.)
?	Show all the shortcuts

Save your robot

💾 **Save .asm** downloads your robot as a file. 📂 **Load .asm files...** brings it back on any computer. You can also drag a file onto the page.

That's how you hand in a robot, or bring it to a friend's computer for a battle!

03 Big ideas to know first

Learn these and the rest gets easy.

Registers: your robot's scratch paper

A **register** is a small box that holds one number. Your robot has 8 of them, named `R0` through `R7`. They all start at 0. Think of them like the variables x and y in math class.



Eight boxes, eight numbers. Here R0 holds 10 and R2 holds 45.

Try it

```
MOV R0, 10     ; put 10 in box R0
ADD R0, 5      ; now R0 holds 15
```

Registers hold **whole numbers** only, so no decimals. Negative numbers are fine. If you ever need more than 8 boxes, your robot also has 256 **memory** slots, written `[0]` to `[255]`.

💬 Comments

Anything after a semicolon `;` is a **comment**. The computer skips it. Comments are notes for humans, so use them to explain your code!

```
SPEED 5    ; full speed ahead!
```

🔖 Labels

A **label** is a name with a colon, like `main:`. It's a bookmark in your code. Jumps use labels to know where to go.

```
main:
    SPEED 5
    JMP main
```

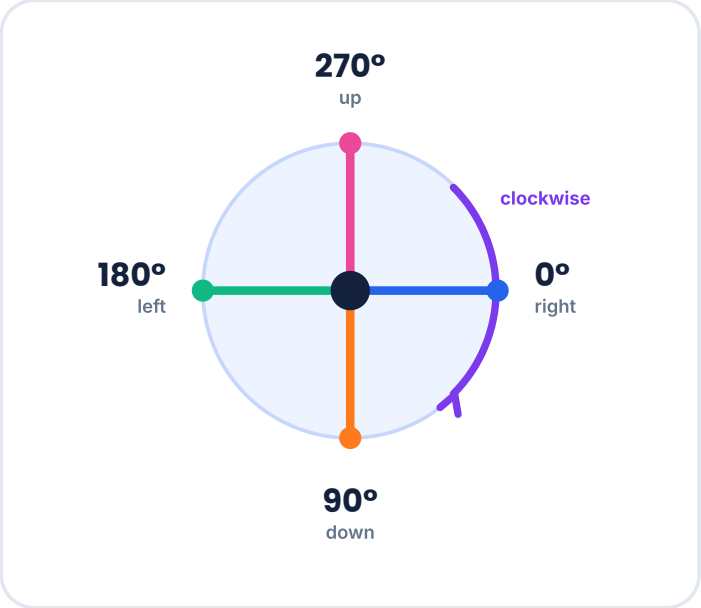
🏷️ Names for numbers

`.def` gives a number a name. It makes your code easier to read, and you only have to change the number in one place.

```
.def WALL_GAP 60
CMP R0, WALL_GAP   ; same as CMP R0, 60
```

Angles: which way is which?

Angles are in degrees, but they work a little differently than in math class. **0° points right**, and the numbers go **clockwise**, like a clock's hands. So 90° points **down**, not up!



🔄 **TURN always takes the shortcut**

`TURN 90` turns right and `TURN -90` turns left. Your robot always turns the shorter way round, so `TURN 270` really turns 90° to the **left**!

Your robot has two parts that turn



🛞 The body

The body is what drives. It turns slowly, at most **8° per tick**.

🎯 The turret

The turret on top turns on its own, up to **20° per tick**. So your robot can drive one way and fire another!

💡 **Orders, not magic**

When you say `SPEED 5`, your robot speeds up a little each tick, like a real car. It keeps following your last order until you give a new one.

04 Every instruction, explained

Your robot's whole vocabulary, in plain English.

Instructions follow a pattern: the **name**, then the things it works on, split by commas. When an instruction changes a value, the answer goes into the **first** thing listed.

```
ADD R0, 5     ; R0 = R0 + 5
;   ^   ^
;   |   +-- the amount to add
;   +------ where the answer goes
```

instruction · register · number · sensor · label · name for a number · comment

Moving numbers DATA

Instruction	What it does	Example	In plain English
`MOV a, b`	Copies b into a	`MOV R1, 90`	*Put 90 into R1.*
`PUSH a`	Puts a on top of the stack	`PUSH R3`	*Save R3 on the pile.*
`POP a`	Takes the top of the stack into a	`POP R3`	*Grab the top of the pile.*

The **stack** is like a pile of plates. You can only add or take from the top. It holds 64 numbers.

Math MATH

Instruction	What it does	Example	In plain English
`ADD a, b`	a = a + b	`ADD R0, 5`	*Add 5 to R0.*
`SUB a, b`	a = a − b	`SUB R0, R1`	*Take R1 away from R0.*
`MUL a, b`	a = a × b	`MUL R0, 3`	*Triple R0.*
`DIV a, b`	a = a ÷ b, no remainder	`DIV R0, 2`	*Cut in half. 7 ÷ 2 gives 3.*
`MOD a, b`	a = the remainder of a ÷ b	`MOD R0, 360`	*Keep R0 from 0 to 359.*
`INC a`	Add 1	`INC R2`	*Count up.*
`DEC a`	Subtract 1	`DEC R2`	*Count down.*
`NEG a`	Flip the sign	`NEG R1`	*5 becomes −5.*
`ABS a`	Make it positive	`ABS R1`	*−20 becomes 20.*
`MIN a, b`	Keep the smaller one	`MIN R0, 45`	*Never above 45.*
`MAX a, b`	Keep the bigger one	`MAX R0, -45`	*Never below −45.*
`RAND a, b`	Random number, 0 to b − 1	`RAND R0, 360`	*Pick any direction.*

⚠️ **Watch out!**
Dividing by zero (`DIV` or `MOD` by 0) crashes your robot's program.

Geometry helpers MATH

Instruction	What it does	Example
`NORM a`	Squeezes an angle into −180 to 179. Shows the shortest way to turn: negative = left, positive = right.	`NORM R0`
`ATAN2 a, dy, dx`	Finds the angle that points toward a spot.	`ATAN2 R0, R2, R1`
`SIN` / `COS`	Sine and cosine × 1000 (no decimals allowed!)	`SIN R1, R0`
`SQRT a, b`	Square root, rounded down	`SQRT R0, R0`

Bonus for curious coders: `AND`, `OR` and `XOR` work on the 1s and 0s (bits) inside a number. Most robots never need them.

Making decisions DECISIONS

This is the most important part of programming. It's how your robot makes choices! It always takes **two steps**:

Step 1: Compare

`CMP a, b` compares two numbers. It doesn't change anything. It just remembers which one is bigger.

Step 2: Jump (maybe)

A jump checks that answer. If it's true, it jumps to a label. If not, it goes to the next line.

Instruction	Jumps when…	Think of it as
`JMP label`	Always	*Go!*
`JE label`	a **equals** b	*If a = b, go.*
`JNE label`	a does **not equal** b	*If a ≠ b, go.*
`JG label`	a is **greater** than b	*If a > b, go.*
`JGE label`	a is **greater than or equal** to b	*If a ≥ b, go.*
`JL label`	a is **less** than b	*If a < b, go.*
`JLE label`	a is **less than or equal** to b	*If a ≤ b, go.*

It's an IF statement: if health is under 30, then run away!

```
1    GET  R0, HEALTH     ; R0 = my health
2    CMP  R0, 30         ; compare my health with 30
3    JL   run_away       ; if health < 30, jump to run_away
4    SPEED 5             ; otherwise, keep charging!
```

Functions and timing FLOW

Instruction	What it does
`CALL label`	Jumps to a function **and remembers where you came from**. Like pausing a video to do a chore, then coming back.
`RET`	Finishes the function and goes back to the line right after the `CALL`. Every `CALL` needs a `RET`.
`WAIT`	"I'm done for this tick." Ends your turn so the game can move everything.
`NOP`	Does nothing ("no operation"). Uses up one coin.
`HALT`	Stops your program for good. Your robot keeps following its last orders.

A chunk of code you `CALL` is called a **function** (some people say *subroutine*). You write it once, like "dodge a projectile," and use it from anywhere.

Controlling the robot ROBOT

Instruction	What it does	Example	In plain English
`GET a, SENSOR`	Reads a sensor into a register	`GET R0, HEALTH`	*How much health do I have?*
`SPEED n`	Speed from −3 (backward) to 5 (full speed)	`SPEED 5`	*Floor it!*
`TURN n`	Turn n degrees from where you face now. + = right, − = left. Takes the shorter way.	`TURN -90`	*Quarter turn left.*
`HEAD n`	Turn to face an exact direction	`HEAD 270`	*Face up.*
`AIM n`	Point the turret at an exact direction	`AIM 0`	*Point the turret right.*
`FIRE`	Fire, if the turret is ready	`FIRE`	*Fire!*
`SCAN width`	Look for the nearest enemy in a cone around the turret	`SCAN 20`	*Look through binoculars.*
`RADAR`	Look for a projectile flying at you (within 250 units)	`RADAR`	*Is anything flying at me?*

After you `FIRE`, the turret needs **15 ticks** to reload. `FIRE` does nothing while it's reloading.

🔦 SCAN is like a flashlight

A wide beam lights up a big area but doesn't reach far. A narrow beam reaches really far, but only in a thin slice.



Memory: 256 lockers 🔐

Your 8 registers are like boxes on your desk: quick to grab, but there are only 8. When you need to **remember something for later**, use **memory**. It's a hallway of **256 lockers**, numbered 0 to 255. Each locker holds one whole number, starts at 0, and keeps its number until you change it.



Square brackets mean "the locker numbered...". Here locker [0] holds 135 and locker [3] holds 42.

Writing and reading a locker

Code	In plain English
`MOV [5], R0`	Put R0's number **into** locker 5.
`MOV R0, [5]`	Copy locker 5's number **into** R0.
`INC [5]`	Add 1 to locker 5. Great for counting!
`MOV R0, [R1]`	R1 holds a locker number. Go to **that** locker.
`MOV R0, [R1+4]`	Go 4 lockers past the one R1 points to.

🏷️ Give your locker a name

Write `.def LAST_SEEN 0` at the top. Then `[LAST_SEEN]` means locker 0, and your code explains itself!

Example: remember where they went

rememberer.asm

```
Rememberer
.def LAST_SEEN 0     ; locker 0
main:
    SCAN 30
    GET  R0, SCAN_DIST
    CMP  R0, 0
    JL   lost        ; nobody seen
    GET  R1, SCAN_ANGLE
    MOV  [LAST_SEEN], R1 ; write it
    AIM  R1
    FIRE
    WAIT
    JMP  main
lost:
    MOV  R1, [LAST_SEEN] ; read it
    ADD  R1, 20          ; look further
    MOV  [LAST_SEEN], R1 ; write it back
    AIM  R1
    WAIT
    JMP  main
```

👀 **Peek inside**

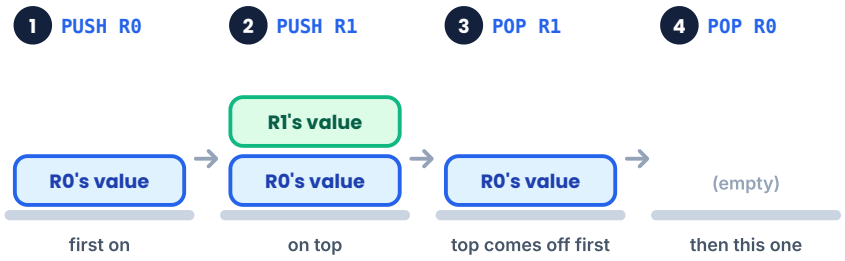
When the match is paused, the CPU inspector shows the lockers your program uses. Point at a **[...]** in your code to see which locker it means right now.

🚫 **Only 0 to 255**

There is no locker 256. `[300]` gets an error before your robot runs. If a register points past 255 during a match, your robot crashes.

The stack: a pile of trays 🍽️

Picture the pile of trays in the cafeteria. You can only put a tray on **top**, and you can only take the **top** one off. That's the **stack**! It holds up to 64 numbers. `PUSH` puts a number on top. `POP` takes the top number off and puts it in a box.



Last on, first off. If you PUSH R0 and then R1, you must POP R1 first, then R0.

Why use it? To save a box and get it back

saver.asm

```
1  Saver
2  main:
3      MOV  R0, 3        ; my speed
4      CALL look         ; do the "look" job
5      SPEED R0          ; R0 is still 3!
6      WAIT
7      JMP  main
8  look:
9      PUSH R0           ; save R0
10     GET  R0, TURRET   ; borrow R0
11     ADD  R0, 15
12     AIM  R0
13     POP  R0           ; R0 back
14     RET               ; go home
```

Saver keeps its speed in R0. The **look** function needs a box too, so it borrows R0. It **saves R0 on the pile** first, then puts it back before going home.

`CALL` uses the pile too! It leaves a **"come back here" note** on top, and `RET` takes the note off to know where to go. Here's the pile while Saver runs (below).



Forgot a POP?

Then `RET` grabs your number instead of its note, and your robot crashes: *RET found a value saved by PUSH.* Every `PUSH` needs a `POP` before `RET`.



Registers `R0` – `R7`
8 boxes. For numbers you're using **right now**.

Memory `[0]` – `[255]`
256 lockers. For things to **remember later**.

The stack
A pile of up to 64. For **saving a box** and finding the way home after `CALL`.

05 Sensors

Your robot's eyes and ears.

Read any sensor with `GET`. For example, `GET R0, FRONT` puts the distance to the wall ahead into R0.

About me

Sensor	What it tells you
`X`, `Y`	Where I am. X goes 0 → 800 (left to right). Y goes 0 → 600 (top to bottom).
`HEADING`	Which way my body is facing (0–359°)
`SPEED`	How fast I'm moving right now
`HEALTH`	How much health I have left (0–100)
`TURRET`	Which way my turret is pointing (0–359°)
`COOLDOWN`	Ticks until I can fire again. 0 means ready!
`FRONT`	How far I can drive straight ahead before I hit something
`MUD`	1 if I'm in mud, 0 if not
`LAST_HIT`	Ticks since a projectile last hit me (−1 if never)

What my SCAN found

Sensor	Tells you
`SCAN_DIST`	How far the enemy is. **−1 = nobody found**
`SCAN_ANGLE`	Which way the enemy is
`SCAN_X` `SCAN_Y`	Enemy's position
`SCAN_HEADING`	Where it's driving
`SCAN_SPEED`	How fast it's going
`SCAN_RANGE`	How far my scan reached

What my RADAR found

Sensor	Tells you
`THREAT_DIST`	How far the projectile is. **−1 = no danger**
`THREAT_ANGLE`	Which way the projectile is
`THREAT_HEADING`	Which way it's flying

About the match

Sensor	Tells you
`TICK`	The match clock
`ENEMIES`	Enemies still alive
`ALLIES`	Teammates alive
`ARENA_W` `ARENA_H`	800 and 600

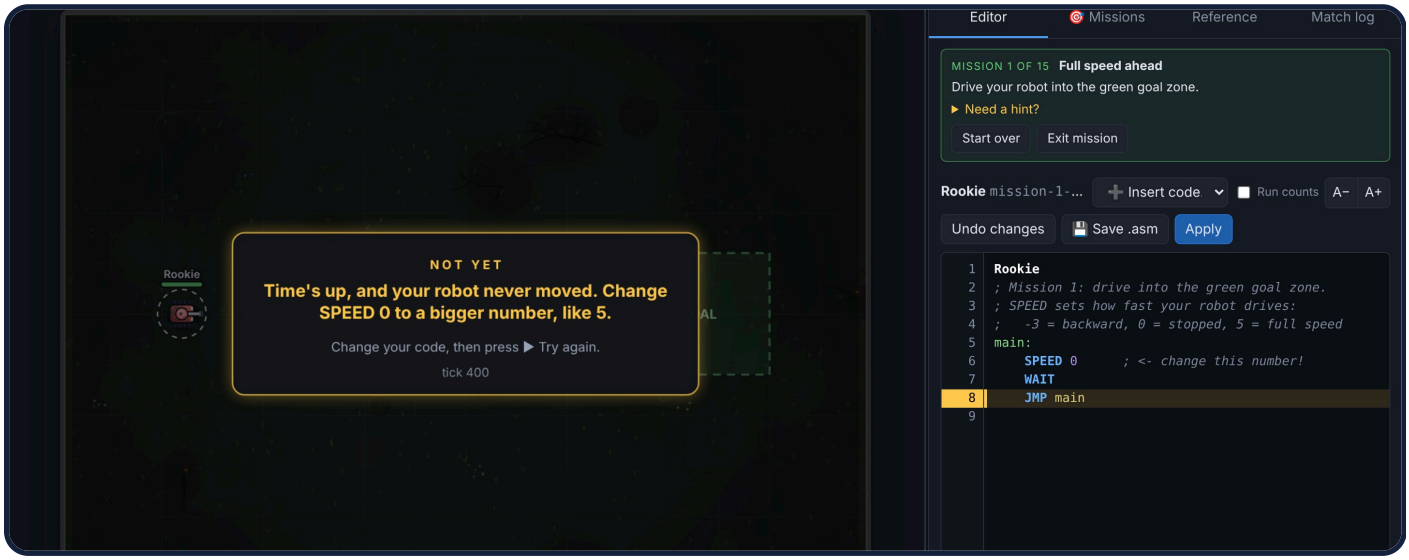
⚠️ **Always check for -1 first!**

If `SCAN_DIST` or `THREAT_DIST` is −1, nothing was found, and the other values are just 0.

06 Missions

15 short challenges inside the app. One idea at a time.

Open the 🎯 **Missions** tab and press **Start** next to a mission. Each mission gives you a robot with starter code that has **one thing to fix**. Fix it, press ▶ Start, and see if you pass!



Mission 1: Rookie's code says `SPEED 0`. Change the number so Rookie drives into the GOAL.

1 Read the goal
It's in the green box.

2 Fix the code
Look for ← *change this*.

3 Press ▶ Start
Watch your robot try.

4 Pass or retry
It tells you what went wrong.

Stuck? Help grows each time you try 🪜

Need a hint?
A small nudge. Open it any time.

→

A bigger hint
After 2 tries. Almost the answer!

→

Show me a working answer
After 4 tries. Type it in yourself!

Your mission map 🗺️

Each unit builds on the one before. Tick a box when you pass a mission!

Unit 1: Getting started

1	**Full speed ahead** SPEED	☐
2	**Which way is up?** HEAD and angles	☐
3	**Hit the target** AIM and FIRE	☐
4	**Find it with SCAN** SCAN, CMP and jumps	☐
5	**Dodge!** RADAR	☐

Unit 2: Registers, loops and functions

6	**Park it** registers, CMP, first jump	☐
7	**Three in a row** changing a register in a loop	☐
8	**Three laps** counting with INC	☐
9	**Come back!** CALL and RET	☐
10	**Boss: dodge and attack** building with functions	☐

Unit 3: Sensors, math and science

11	**Don't crash** sensor loops with FRONT	☐
12	**Follow the leader** IF / ELSE	☐
13	**Mission control** coordinates and ATAN2	☐
14	**Lead the target** experimenting	☐
15	**Graduation match** putting it all together	☐

Good to know

- Changing your code starts the attempt over, so old mistakes never count.
- Click **Exit mission** (or a robot card) to go back to your own robots.
- Your progress is saved in **this browser**.

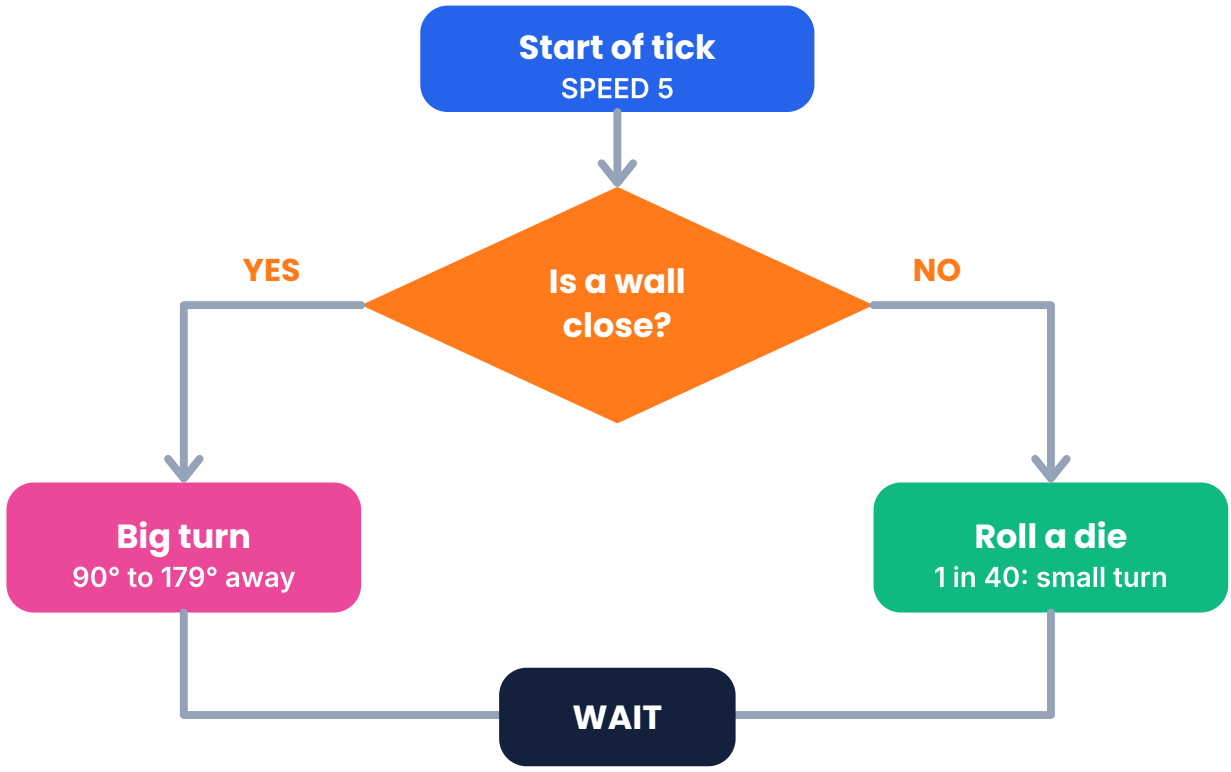
Missions + this guide

The missions teach the ideas. This guide is your reference whenever you want the details. When you finish all 15, read Wanderer next, then take on the **Challenges**!

07 Let's read a real robot

Meet Wanderer. It just drives around the arena.

Wanderer turns away from walls, and sometimes it changes direction at random so it doesn't get boring. Before reading the code, look at this picture of what it "thinks" about every tick:



Wanderer's brain, one tick at a time

🧠 **The big idea**

Every tick, Wanderer asks one question: **Is a wall close?** If yes, it makes a big turn. If no, it rolls a 40-sided die. On a 0, it makes a small turn. That's the whole robot!

💡 **Try it yourself**

Type in Wanderer (next page) and press ▶ Start. Then press **Step** a few times and watch the yellow marker move through the lines. In section 08 you'll make Wanderer freeze the moment a wall gets close!

Line by line

The line numbers in the code match the numbers in the table.

wanderer.asm

```
1  Wanderer
2  .shape circle
3  .drive wheels
4  .turret short
5
6  .def WALL_GAP 60
7
8  main:
9      SPEED 5
10     GET  R0, FRONT
11     CMP  R0, WALL_GAP
12     JG   wander
13     RAND R1, 90
14     ADD  R1, 90
15     TURN R1
16     JMP  end
17 wander:
18     RAND R1, 40
19     CMP  R1, 0
20     JNE  end
21     RAND R1, 120
22     SUB  R1, 60
23     TURN R1
24 end:
25     WAIT
26     JMP  main
```

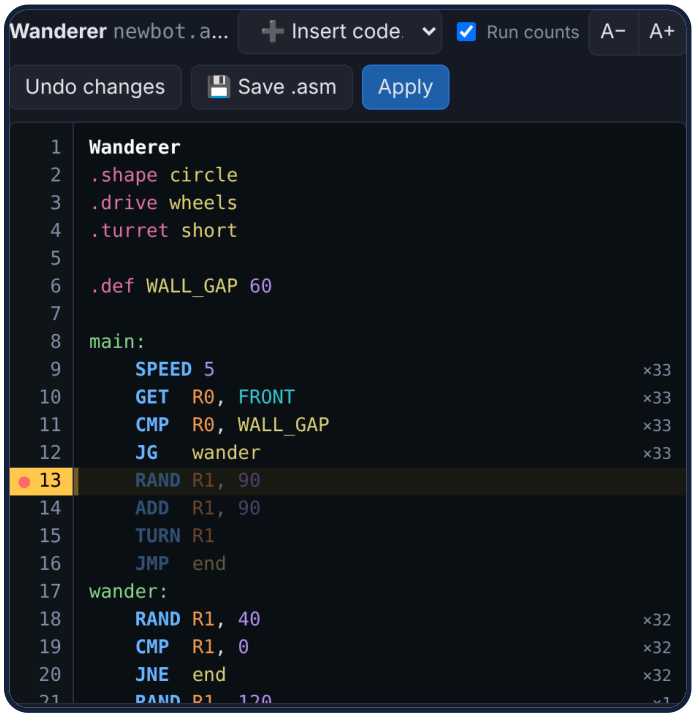
Line	What it means
1	The robot's name.
2–4	How the robot looks. Just for style!
6	"WALL_GAP" now means 60.
8	Bookmark: the loop starts here.
9	Drive at full speed.
10	How far until I hit a wall? Put it in R0.
11	Compare that distance with 60.
12	More than 60 away? All good! Jump to **wander**.
13	Wall is close! Pick a number from 0 to 89.
14	Add 90. Now R1 is 90 to 179.
15	Make that big turn, away from the wall.
16	Already turned, so skip to **end**.
17	Bookmark for the random part.
18–20	Roll a 40-sided die. Not a 0? Skip to **end**.
21–22	Rolled a 0! Pick a small turn, −60 to 59.
23	Make that small turn.
24	Bookmark: the end of the loop.
25	Done for this tick. Let the game move me.
26	Back to the top for the next tick!

08 Debug like a pro

Find out exactly what your robot is thinking.

Is your robot doing something weird? Don't guess. Pause it on the exact line you care about and read what it's thinking.

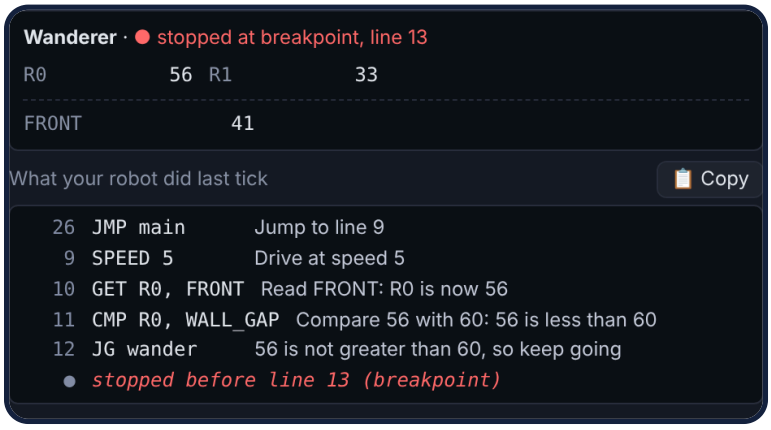
Breakpoints: freeze on any line 🔴



1. **Click a line number.** A red dot appears. Here it's line 13, the "wall is close" part of Wanderer.
2. **Press ▶ Start.** The match pauses just *before* that line runs.
3. **Read the inspector** (below) to see why your robot got there. It also shows memory and the stack when you use them.
4. **Press Step or Start** to carry on. Click the dot again to remove it.

> 🔢 **Tick "Run counts"**
> Numbers like **×33** show how many times each line ran. Lines that never ran turn dim, so a function that never got called is easy to spot!

"What your robot did last tick"



Whenever the match is paused, the CPU inspector explains every line your robot ran, in plain English. Stuck? Press 📋 **Copy** and paste the list into a message to ask for help.

Wanderer is 56 units from a wall. 56 is not greater than 60, so it doesn't jump. Next it will turn!

> 👀 **Watch the arena too**
> For the robot you selected, a light-blue arrow shows where its body is turning and a dotted line shows where its turret is aiming. When paused, a label spells it out, like **speed 0 → 5, turning to 80°**.

Oops! Common mistakes

Every programmer makes these. The editor catches many of them for you.

1 Forgot a comma
`MOV R0 5` needs a comma: `MOV R0, 5`. The editor spots this and tells you.

2 Label without a colon
A label needs its colon: `main:`, not `main`.

3 Math-class equals
`R0 = 5` isn't assembly. Write `MOV R0, 5` to put 5 in a box.

4 Forgetting WAIT
Without `WAIT`, your loop runs until all 50 coins are gone. Put `WAIT` at the end of your main loop.

5 Jumping without CMP
Jumps only check the **last** `CMP`. Math like `ADD` doesn't count. Compare right before you jump.

6 Using an empty scan
If `SCAN_DIST` is −1, nobody was found. Check for −1 first and skip the firing part.

7 Mixing up angles
0° is right, 90° is **down**, and angles go clockwise.

8 CALL without RET
Every function you `CALL` must end with `RET`. And every `PUSH` needs a `POP` before that `RET`.

9 Dividing by zero
The editor catches `DIV R0, 0`. But if a box holds 0 when `DIV` runs, your program crashes.

10 JMP 3 isn't line 3
A number after a jump means instruction 3, not line 3. Always jump to a **label**.

Red means stop, yellow means look

```
1  Oopsie
2  main:
3      SPEED 10
✗ 4    MOV  R0 5
5      WAIT
6      JMP  main
7
```

Line 4 MOV needs 2 values, like "MOV R0, 5", but this line gives 1. Did you forget a comma? Write "MOV R0, 5".

Line 3 "SPEED 10" drives at 5: SPEED goes from -3 to 5.

🟥 Errors

Your code can't run until you fix them. Click an error to jump right to that line. The message tells you how to fix it.

🟨 Warnings

Your code still runs, but something might surprise you. For example, `SPEED 10` only drives at 5, because 5 is the top speed.

💥 Crashes

"(crashed)" shows after its name and the crashing line turns red. With Simple view on, the inspector explains why in plain words, like “it tried to divide by zero.”

09 Make it yours

Give your robot its own look. Then get ready for the final battle!

Every new robot starts with **nine look settings** right under its name. Change the word after each one to style your robot. Looks never change how your robot plays, so pick whatever you like!



Blaze, Bubbles, Volt and Checkmate: same code, four different looks. Spot the star, plasma, rocket and laser shots!

blaze.asm

```
Blaze
.shape wedge
.drive tracks
.turret twin
.color orange
.decal flames
.badge 7
.trail sparks
.shot rocket
; your code starts here
main:
    WAIT
    JMP  main
```

The nine look settings

Setting	Choices	What it changes
`.shape`	tank, circle, hexagon, wedge	The body shape
`.drive`	tracks, wheels, hover	What it rolls on
`.turret`	standard, short, twin	The turret on top
`.color`	red, blue, yellow, green, purple, orange, cyan, pink, white, gray, lime, indigo, brown	Paint color (**auto** lets the game pick)
`.decal`	stripes, flames, lightning, spots, checker	A paint job on the body
`.badge`	star, bolt, heart, crown, paw, shield, or a number 1 to 99	A badge next to your name
`.trail`	smoke, sparks, bubbles, rainbow	Left behind as you drive
`.boom`	fire, confetti, pixels, fireworks	How it explodes when destroyed
`.shot`	shell, laser, plasma, rocket, star	How its projectiles look

The rules: put the settings right under the name, before any code. One word each, no commas, each setting only once. Made a typo, like `.shape triangle`? The editor shows an error that lists the choices.

The final battle 🏆

At the end, everyone's robot goes into the same arena. Here's how to get yours in.



Five robots, five programs, one arena. Badges show next to each name, so you can find yours fast.

1. **Pick a unique name.** Line 1 is your robot's name. Make it one nobody else has!
2. **Save it.** Click 💾 Save .asm (or Ctrl + S). You get a .asm file.
3. **Hand it in.** Your teacher loads every robot into one arena.
4. **Battle!** Watch your code take on everyone else's.

How the battle works

- The last robot standing wins.
- If time runs out (6000 ticks), the robot with the most health wins.
- Two robots picked the same `.color` ? The first one in the list keeps it. The other gets the next free color.
- In team battles, your projectiles can't hurt teammates, and teams get team colors.

✅ Before you hand it in

- [ ] Line 1 is a name nobody else has.
- [ ] No red errors under the editor.
- [ ] You tested it against an example robot (**+ Add example**).
- [ ] You saved it with 💾 Save .asm.

🎯 **Follow your own shots**

Click your robot's card to select it. With **Highlight mine** on (it is, unless you change it in ⚙ Settings), your projectiles get a white ring and everyone else's look fainter.

10 Challenges

Start from Wanderer and change it step by step. Test after every change!

CHALLENGE 1 ★☆☆☆☆

☐ Speed demon

Change `WALL_GAP` to 120. What happens? Now try 20.

CHALLENGE 2 ★☆☆☆☆

☐ Slow and steady

Make Wanderer drive at speed 2 instead of 5.

CHALLENGE 3 ★★☆☆☆

☐ Always turn right

Change the wall turn so it's always exactly 90°.

CHALLENGE 4 ★★★☆☆

☐ Reverse gear

When a wall is close, back up (`SPEED -3`) instead of turning.

CHALLENGE 5 ★★★☆☆

☐ Look around

Add `SCAN 30` and make the turret spin 15° every tick.

CHALLENGE 6 ★★★★☆

☐ Lock on and fire

If `SCAN_DIST` is not −1, `AIM` at `SCAN_ANGLE` and `FIRE` .

CHALLENGE 7 ★★★★☆

☐ Panic mode

If `HEALTH` drops below 30, go full speed and turn more often.

CHALLENGE 8 · BOSS LEVEL ★★★★★

☐ Projectile dodger

Use `RADAR` to dodge projectiles. Look at the Dodger example robot for ideas!

How to test a challenge

1. **Copy Wanderer** and give it a new name on line 1.
2. **Change one thing** Just one! Small steps are easier to fix.
3. **Press ▶ Start** Does it do what you wanted?
4. **Not working?** Set a breakpoint and read the last tick.

➕ **Head start**

The ➕ **Insert code** menu has **Scan and fire**, **Wall check** and **Dodge projectiles** patterns. Use them as building blocks!

✅ **Check them off**

Tick the box on each card when your robot does it. Stuck? The answer key starts on the next page, but give it a real try first!

11 Answer key

Every challenge, solved. Green lines are new or changed.

💡 **How to read this**

Each answer shows what to change and why it works. Lines with a **green bar** are the new or changed lines. Every program here was tested in the real game.

1 Speed demon

Change one number

What to change

Change line 6 from `.def WALL_GAP 60` to 120, then try 20.

Why it works

With 120, Wanderer starts turning sooner, so it stays farther from walls. With 20, it waits until the last second, can't turn fast enough, and bumps walls a lot. Bumps cost health!

BEFORE

```
.def WALL_GAP 60
```

→ AFTER

```
.def WALL_GAP 120
```

2 Slow and steady

Change one number

What to change

Change `SPEED 5` to `SPEED 2`.

Why it works

A slower robot almost never crashes into walls, because it has time to turn. The downside? It's an easier target!

BEFORE

```
main:
    SPEED 5
```

→ AFTER

```
main:
    SPEED 2
```

3 Always turn right

Swap three lines for one

What to change

Replace the three lines that pick a random turn with `TURN 90`.

Why it works

Positive turns go right. The robot asks for a 90° right turn each tick until the wall is out of the way.

BEFORE

```
    RAND R1, 90
    ADD  R1, 90
    TURN R1
```

→ AFTER

```
    TURN 90
```

4 Reverse gear

Use a register as a countdown timer

What to change

When a wall is close, put 18 in `R2`. While R2 is bigger than 0, drive backward and count down by 1 each tick.

Why it works

If you only back up, you'll drive right back into the same wall! So the robot also turns while it reverses. When the countdown hits 0, it drives forward again.

reverser.asm

```
Reverser
.def WALL_GAP   60
.def BACK_TICKS 18  ; back-up time
main:
    CMP  R2, 0      ; counting down?
    JG   backing    ; yes: back up
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    MOV  R2, BACK_TICKS ; start!
    RAND R1, 90
    ADD  R1, 90
    TURN R1         ; turn as we go
    JMP  end
backing:
    SPEED -3        ; reverse!
    DEC  R2         ; count down
    JMP  end
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  end
    RAND R1, 120
    SUB  R1, 60
    TURN R1
end:
    WAIT
    JMP  main
```

5 Look around

Add a turret section

What to change

Add a `targeting:` section before `end:`. Change the two `JMP end` / `JNE end` jumps to go to `targeting` so the turret code runs every tick.

Why it works

Each tick, the robot reads where its turret points, adds 15°, and aims there. The turret can turn 20° per tick, so 15° is easy. Round and round it goes!

lookout.asm

```
Lookout
.def WALL_GAP 60
main:
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  targeting
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  targeting
    RAND R1, 120
    SUB  R1, 60
    TURN R1
targeting:
    SCAN 30         ; look around
    GET  R3, TURRET ; where's my turret?
    ADD  R3, 15     ; 15° more
    AIM  R3         ; spin!
end:
    WAIT
    JMP  main
```

6 Lock on and fire

Fire when you see someone

What to change

After `SCAN 30`, check `SCAN_DIST`. If it's −1, keep spinning. Otherwise aim at `SCAN_ANGLE` and fire.

Why it works

`JL sweep` jumps when SCAN_DIST is less than 0, which only happens when nobody was found. If someone *was* found, we skip right past it to the aiming code.

spotter.asm

```
Spotter
.def WALL_GAP 60
main:
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  targeting
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  targeting
    RAND R1, 120
    SUB  R1, 60
    TURN R1
targeting:
    SCAN 30
    GET  R3, SCAN_DIST
    CMP  R3, 0
    JL   sweep       ; -1 = nobody
    GET  R3, SCAN_ANGLE ; which way?
    AIM  R3          ; aim at them
    FIRE             ; fire!
    JMP  end
sweep:
    GET  R3, TURRET
    ADD  R3, 15
    AIM  R3          ; keep spinning
end:
    WAIT
    JMP  main
```

7 Panic mode

Use registers to hold settings

What to change

Keep the speed in `R4` and the dice size in `R5`. Normally use speed 3 and a 40-sided die. If health is under 30, switch to speed 5 and a 10-sided die.

Why it works

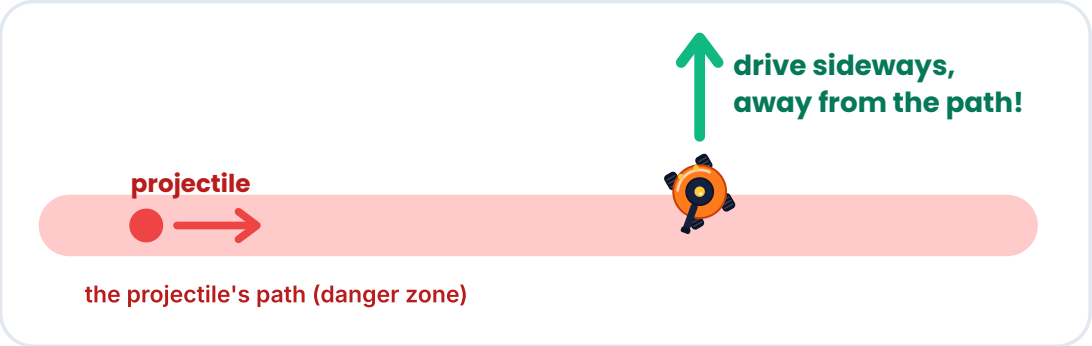
A 10-sided die lands on 0 much more often than a 40-sided one, so the robot turns 4 times as often. `SPEED R4` and `RAND R1, R5` use whatever is in the boxes!

panicker.asm

```
Panicker
.def WALL_GAP 60
main:
    MOV  R4, 3       ; normal speed
    MOV  R5, 40      ; 40-sided die
    GET  R0, HEALTH
    CMP  R0, 30
    JGE  calm        ; 30+? stay calm
    MOV  R4, 5       ; PANIC! go fast
    MOV  R5, 10      ; 10-sided die
calm:
    SPEED R4
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  targeting
wander:
    RAND R1, R5      ; roll the die
    CMP  R1, 0
    JNE  targeting
    RAND R1, 120
    SUB  R1, 60
    TURN R1
targeting:
    SCAN 30
    GET  R3, SCAN_DIST
    CMP  R3, 0
    JL   sweep       ; -1 = nobody
    GET  R3, SCAN_ANGLE ; which way?
    AIM  R3          ; aim at them
    FIRE             ; fire!
    JMP  end
sweep:
    GET  R3, TURRET
    ADD  R3, 15
    AIM  R3          ; keep spinning
end:
    WAIT
    JMP  main
```

8 Projectile dodger

Boss level: RADAR plus a subroutine



boss.asm

```
Boss
.def WALL_GAP 60
main:
    RADAR              ; projectiles coming?
    GET  R0, THREAT_DIST
    CMP  R0, 0
    JL   roam          ; -1 means no danger
    CALL dodge         ; danger! go dodge
    JMP  targeting
roam:
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  targeting
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  targeting
    RAND R1, 120
    SUB  R1, 60
    TURN R1
targeting:
    SCAN 30
    GET  R3, SCAN_DIST
    CMP  R3, 0
    JL   sweep         ; -1 = nobody
    GET  R3, SCAN_ANGLE ; which way?
    AIM  R3            ; aim at them
    FIRE               ; fire!
    JMP  end
sweep:
    GET  R3, TURRET
    ADD  R3, 15
    AIM  R3            ; keep spinning
end:
    WAIT
    JMP  main
```

What to change

Start each tick with `RADAR`. If a projectile is coming, `CALL dodge`. If not, wander like normal. Either way, run the targeting code.

Why it works

The dodge subroutine works out which side of the projectile's path the robot is on. Then it drives sideways, even farther to that side, so the projectile misses.

boss.asm (continued, at the very bottom)

```
dodge:
    GET  R1, THREAT_ANGLE
    ADD  R1, 180       ; from projectile to me
    GET  R2, THREAT_HEADING
    SUB  R1, R2        ; left or right of its path?
    NORM R1
    CMP  R1, 0
    JL   dodge_left
    ADD  R2, 90        ; on the right: go right
    JMP  dodge_go
dodge_left:
    SUB  R2, 90        ; on the left: go left
dodge_go:
    HEAD R2            ; face sideways
    SPEED 5            ; and go!
    RET                ; back to the CALL
```

You did it!

Your robot can roam, scan, fire, panic and dodge. Now take on Mission 15, the Graduation match, then invent your own strategy and battle your friends!